

Curación pendiente

Los 41 módulos que el grafo no cubre, con todo lo verificable ya resuelto.

Generado por `scripts/hoja_de_curacion.py` · se regenera, no se edita a mano

No queda nada pendiente

Los **10** módulos que la medición sigue marcando «sin nodo» están todos decididos. Nueve son marca **T** —ni objetos ni flechas, así que no entran— y uno ya está cubierto por un nodo que existía. Siguen apareciendo como «sin nodo», y es correcto: no hay nodo. Lo que no son es trabajo.

módulo	decisión
<code>AlgebraicTopology.SimplexCategory</code> .	T · es <code>SimplexCategory</code> presentada por generadores: la misma categoría con otro nombre, y dos nombres para un objeto gastan dos plazas
<code>GeneratorsRelations.Basic</code>	T · es el teorema de Akra-Bazzi y sus piezas de demostración, no un objeto: el mismo caso que <code>prime-factorization</code>
<code>Computability.AkraBazzi</code> .	T · interfaz de efectos de Lean: endofuntores sobre <code>Type</code> , y un lazo no es una dependencia
<code>SumTransform</code>	T · <code>Part.fix</code> es el punto fijo con el que Lean define funciones parciales: maquinaria de definición
<code>Control.Bitraversable.Basic</code>	T · la misma rama de efectos
<code>Control.Fix</code>	T · la misma rama de efectos
<code>Control.Functor.Multivariate</code>	T · vectores de tipos para inductivos multivariados: infraestructura
<code>Control.Monad.Cont</code>	T · un <code>PFilter</code> es un filtro sobre un preorden, pero ‘ <code>Order</code> ’ no es área del grafo y un nodo solo no la justifica: sin padre la arista no existiría. Primer candidato si algún día entra la teoría de ordenes
<code>Data.TypeVec</code>	T · <code>ONote</code> y <code>NONote</code> son la forma normal de Cantor como dato computable: notación, no objeto. El nodo ‘ <code>ordinals</code> ’ ya existe
<code>Order.PFilter</code>	CUBIERTO · ‘ <code>TopCat</code> ’ ya es la identidad de ‘ <code>point-set-topology</code> ’. La convención del proyecto pone el envoltorio categorico en el campo ‘ <code>lean</code> ’ del concepto (<code>group-theory</code> lleva <code>GrpCat</code>), no en un nodo aparte
<code>SetTheory.Ordinal.Notation</code>	
<code>Topology.Category.TopCat.Basic</code>	

Si mañana la medición destapa módulos nuevos, esta hoja vuelve a llenarse sola.

1 · La marca. Es lo que sostiene el grafo, y no la decide nada automático:

C	una categoría	→	vértice
S	una subcategoría plena	→	vértice , y la inclusión es arista
F	un funtor o clase de flechas	→	arista <i>en este ambiente</i>
O	un objeto individual	→	vértice degenerado
T	ni objetos ni flechas	→	fuera

F **no** dice «esto no puede ser un vértice nunca». Eso es falso, y está demostrado que lo es en `FlechasComoObjetos.lean`: las flechas de C son exactamente los objetos de la categoría de flechas `Arrow C`, y los funtores son los objetos de la categoría de funtores $[C, D]$. Lo que dice es que **en este grafo** —cuyos objetos son conceptos y cuyas flechas son dependencias— la etiqueta nombra una flecha.

`homology` es **F**: aquí no es una colección que se pueda colimitar, es el funtor *a lo largo del cual* se colimita. `prime-factorization` es **T**: es un teorema, no un objeto.

2 · El padre. De qué concepto es especialización. La flecha va del general al específico.

3 · Cuáles nombres se quedan. Que exista no lo hace correcto: `Nat.Prime` es la noción de primo, `Nat.minFac` no lo es.

Lo que **no** hay que decidir, porque ya está verificado: si el nombre existe, en qué módulo vive, cuál es el canónico —las citas— y el DAG de `imports`. Los identificadores de abajo están **leídos de su declaración**, no deducidos de la ruta: de 447 deducidos así, 95 no existían.

Antes de dar por buena una tanda

```
python -m scripts.recuperacion_contra_proofnet
```

Precisión y cobertura contra 371 formalizaciones de oro, con su modelo nulo y sin gastar API.

Baseline hoy: 22,8 % / 18,0 % contra 1,45 % / 3,3 % — 15,7×. Si la precisión baja, esa tanda **no entra**.

Ya pasó: ofrecer los sustantivos de los nodos generados bajaba de 14,0 % a 11,5 %. Añadir vocabulario tiene coste.

Y la regla tiene letra pequeña. La primera tanda de 31 nodos bajaba la precisión a 19,9 % sin mover la cobertura, y la causa no era ningún veredicto equivocado —los 47 nombres los acepta **#check**— sino que el emparejador tokeniza el **id y el nombre** de cada nodo: **different-ideal** aportaba el token **different**, que sale en media biblioteca, y con eso gastaba una de las dos plazas del prompt. La puerta que lo arregla —*sólo se ocupa plaza con una keyword declarada*— subió la línea base de 21,3 % a 22,8 %. Si una tanda baja la precisión, mira primero si sus nodos entran en plaza por su nombre.